

SonarQube

for DevOps Beginners

A PRACTICAL BEGINNER'S GUIDE TO
CODE QUALITY IN DEVOPS



ANALYZE CODE



IMPROVE QUALITY



REDUCE RISKS



ACCELERATE
DELIVERY

- ✓ STEP-BY-STEP SETUP
- ✓ CI/CD INTEGRATION
- ✓ QUALITY GATES
- ✓ BEST PRACTICES
- ✓ TROUBLESHOOTING
- ✓ REAL DEVOPS WORKFLOWS



YouTube



GitHub



LinkedIn



X



Instagram



Telegram



VERIQTA

1. WHAT SONARQUBE ACTUALLY DOES



WHAT PROBLEM SONARQUBE SOLVES

SonarQube helps teams find code quality problems early, before they reach production. It detects bugs, security vulnerabilities, code smells, duplicated code and maintainability issues automatically.



STATIC CODE ANALYSIS IN SIMPLE TERMS

SonarQube reads your source code without executing it. It analyzes structure, patterns and logic against thousands of rules to find issues that can cause bugs, security risks or future maintenance problems.



SONARQUBE'S PLACE IN DEVOPS

SonarQube fits into the "Shift-Left" approach. It gives fast feedback during development and in CI pipelines so teams can fix issues early, deliver higher quality code and reduce technical debt.

CODE QUALITY VS APPLICATION TESTING



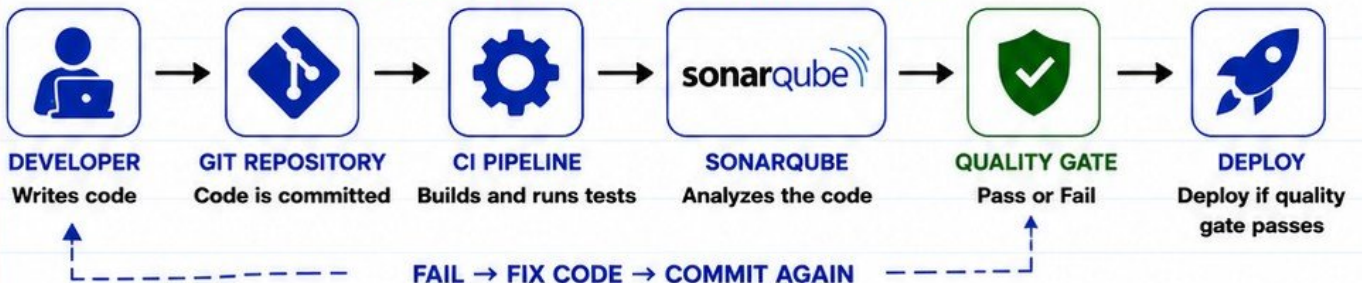
- Code Quality (SonarQube) checks the code you write.
- Application Testing checks the behavior of the running application.
- Both are essential. SonarQube is not a tester.

SONARQUBE VS CI/CD TOOLS



- CI/CD tools build, test and deploy your application.
- SonarQube analyzes code quality and security.
- CI/CD is HOW you deliver. SonarQube is HOW you keep code clean and safe.

BASIC FLOW: WHERE SONARQUBE FITS



BEGINNER NOTE

SonarQube analyzes code. It does **NOT** replace unit tests, integration tests, system tests or runtime monitoring. Use SonarQube together with your testing strategy.

2. UNDERSTANDING SONARQUBE ARCHITECTURE

KEY COMPONENTS



SONARQUBE SERVER

The main application that stores, analyzes and shows results.



DATABASE

Stores all configuration data, projects and analysis results.



COMPUTE ENGINE

Handles the heavy analysis work and processes the source code.



SEARCH ENGINE

Indexes the data so results can be searched quickly.



SONARSCANNER

Runs on your machine or CI server to analyze source code and send results to SonarQube.



WEB INTERFACE

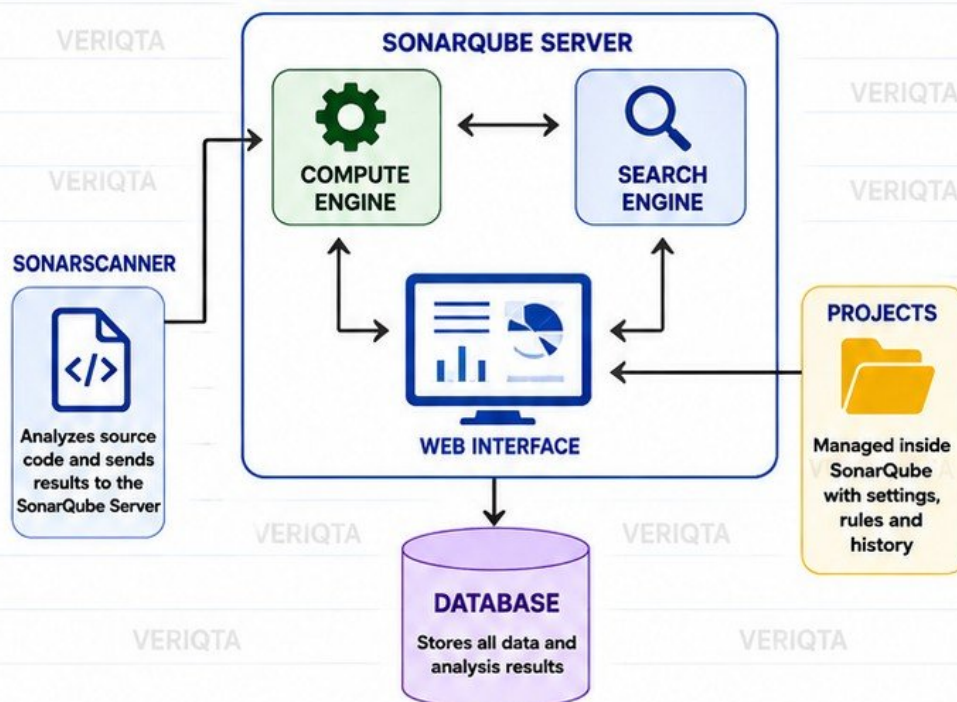
Where users browse projects, view metrics, issues and quality gates.



PROJECTS

Containers for your source code, settings, rules and analysis history.

SONARQUBE INTERNAL ARCHITECTURE



HOW ANALYSIS RESULTS REACH SONARQUBE

- 1 Developer writes code and pushes to Git.
- 2 SonarScanner runs (locally or in CI) and analyzes the source code.
- 3 Scanner sends the analysis report to SonarQube Server.
- 4 Compute Engine processes the report and stores data in the Database.
- 5 Search Engine indexes the data for fast searching.
- 6 Results are displayed in the Web Interface and evaluated by Quality Gates.



SIMPLE ARCHITECTURE FLOW



BEGINNER NOTE

This architecture allows SonarQube to analyze large codebases, store history safely and give fast feedback to developers and DevOps teams.

[YouTube](#)[GitHub](#)[LinkedIn](#)[X](#)[Instagram](#)[Telegram](#)

3. INSTALLING SONARQUBE FOR PRACTICE

1. BASIC LAB REQUIREMENTS

-  **OS:** Windows / macOS / Linux
-  **RAM:** Minimum 4 GB (8 GB recommended)
-  **Disk:** At least 10 GB free space
-  **Docker:** Latest version installed
-  **Internet:** Required to pull Docker image

2. JAVA REQUIREMENTS



- SonarQube requires Java.
- Minimum:** Java 11
- Recommended:** Java 17 (LTS)
- Check your Java version:

```
java -version
```

 Make sure **JAVA_HOME** is set correctly.

3. RUNNING SONARQUBE LOCALLY (DOCKER-BASED SETUP)



Run the SonarQube Community Edition

```
docker run -d --name sonarqube \
  -p 9000:9000 -p 9092:9092 \
  sonarqube:lts-community
```

What this command does:

- ✓ Pulls the SonarQube image
- ✓ Runs SonarQube in a container
- ✓ Exposes ports 9000 and 9092
- ✓ Starts the service in the background

4. REQUIRED PORTS

PORT	PURPOSE
9000	SonarQube Web Interface
9092	Internal Communication



Note:
Make sure your firewall allows these ports.

5. ACCESSING THE WEB INTERFACE



Open your browser and go to:

```
http://localhost:9000
```

You should see the SonarQube login page.

6. FIRST LOGIN



Default Credentials:

Username: admin
Password: admin

You will be prompted to change the password.

7. VERIFYING THAT SONARQUBE IS RUNNING



Container Status
Run:

```
docker ps
```

You should see the sonarqube container running.



Open Dashboard

Go to `http://localhost:9000` and login.

Dashboard should load without errors.



System Status

Go to:
Administration > System
Status should be **GREEN**.



Background Tasks

Go to:
Administration > Background Tasks
All tasks should be completed successfully.



COMMON MISTAKE

Treating the embedded database as a production database.
The embedded database is for learning and testing only.
For production, use an external PostgreSQL database.



4. YOUR FIRST SONARQUBE PROJECT



Step-by-step guide to create your first project and run analysis

1 CREATE A PROJECT

- Go to Projects > Create Project
- Choose Manual
- Give your project a name
- Click Create

2 PROJECT KEY

- A unique identifier for your project
- Example: **sample-app**
- Use simple, lowercase, no spaces

3 GENERATE ANALYSIS TOKEN

- Go to My Account > Security
- Click Generate Tokens
- Give it a name
- Copy and save the token
- Use this token in your scanner
- Tokens are safer than passwords

4 CONNECT LOCAL REPO

- Ensure your code is in Git (local or remote)
- Example:

```
git clone https://github.com/you/repo.git
```

- Navigate to your project folder

EXAMPLE: GENERATED TOKEN

```
sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

- Use in CI/CD pipelines
- Use in SonarScanner
- Keep it private and secure
- Rotate tokens regularly

5 RUN THE FIRST ANALYSIS

- Install SonarScanner
- Run the command:

```
sonar-scanner \
-Dsonar.projectKey=sample-app \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.login=YOUR_TOKEN
```

6 FIND THE RESULTS

- Go to Projects
- Select your project
- Check the Dashboard

7 WHAT YOU WILL SEE

- Overall Quality Grade
- Bugs
- Vulnerabilities
- Code Smells
- Coverage (if available)
- Duplications

8 YOU DID IT!

Your first SonarQube analysis is complete. Keep improving your code quality!

BEGINNER WORKFLOW: HOW IT ALL CONNECTS



IMPORTANT NOTE

This is the first step in your DevOps quality workflow. Always run analysis before deployment to catch issues early and deliver better software.



YouTube



GitHub



LinkedIn



X



Instagram



Telegram

5. UNDERSTANDING SONARSCANNER

1 WHAT SONARSCANNER DOES



- Analyzes your source code.
- Detects bugs, vulnerabilities, code smells.
- Measures code quality, duplication and coverage.
- Sends the results to SonarQube Server.

2 SCANNER VS SONARQUBE SERVER

SONARSCANNER



- Runs on your machine or CI server.
- Analyzes the code.
- Sends results to SonarQube Server.

SONARQUBE SERVER



- Central server.
- Stores, processes and displays results.
- Provides dashboards and quality gates.

VS

3 INSTALLING SONARSCANNER



- Download from the official SonarQube website.
- Extract the file.
- Add the /bin folder to your system PATH.
- Verify installation:

```
sonar-scanner --version
```

4 sonar-project.properties

This file tells the scanner what to analyze and how to connect.

```
# Project identification
sonar.projectKey=my-app
sonar.projectName=My Application
sonar.projectVersion=1.0

# Sources
sonar.sources=src

# Language
sonar.language=java

# Encoding
sonar.sourceEncoding=UTF-8
```

5 sonar.projectKey



- Unique identifier for your project.
- Must be the same in every scan.
- Example:

```
sonar.projectKey=my-app
```

6 sonar.sources



- Path to your source code directories.
- Can be a single folder or multiple paths.
- Example:

```
sonar.sources=src,lib
```

7 AUTHENTICATION



- Use a token, not your password.
- Generate token in SonarQube: My Account > Security > Generate Tokens.
- Pass token to the scanner:

```
sonar.login=YOUR_TOKEN_HERE
# or use environment variable
SONAR_TOKEN=YOUR_TOKEN_HERE
```

8 RUNNING SONAR-SCANNER



- Open terminal in your project directory.
- Run the scanner:

```
sonar-scanner
```

- Scanner reads the configuration, analyzes code and sends results to SonarQube Server.

9 READING SCANNER OUTPUT



- Look for "ANALYSIS SUCCESSFUL".
- Key information in the output:
 - ✓ Project key
 - ✓ Server URL
 - ✓ Quality Gate status
 - ✓ Link to dashboard

- Example success message:

```
ANALYSIS SUCCESSFUL, you can browse
http://localhost:9000/dashboard?id=my-app
```

10 TROUBLESHOOTING A FAILED SCAN

PROBLEM	POSSIBLE CAUSE	HOW TO FIX
✗ Authentication failed	Invalid or missing token	Check token, regenerate if needed, and set sonar.login or SONAR_TOKEN
✗ Could not connect to server	Wrong URL or server not reachable	Verify sonar.host.url and ensure SonarQube Server is running
✗ No such file or directory	Incorrect sonar.sources path	Check path in sonar-project.properties and ensure files exist
✗ Java not found	Java not installed or not in PATH	Install Java 11 or 17 and set JAVA_HOME
✗ Analysis failed with exception	Permissions, memory or plugin issues	Check logs, increase memory (-Xmx2g), update plugins if required



BEGINNER TIP: Always check the full error message in the scanner output. It usually tells you exactly what went wrong and how to fix it.

6. BUGS, VULNERABILITIES AND CODE SMELLS



WHAT SONARQUBE REPORTS

SonarQube analyzes your source code and reports issues that can affect the quality, security and maintainability of your software.



BUGS

Real or highly likely errors that can cause your application to crash, behave unexpectedly or produce incorrect results.



VULNERABILITIES

Weaknesses in your code that attackers could exploit to steal data, bypass controls or cause damage.



CODE SMELLS

Maintainability issues that are not bugs yet, but make code harder to understand, change or extend.



SECURITY HOTSPOTS

Code that looks risky and needs human review to decide if it is actually vulnerable or safe.

WHY THESE CATEGORIES ARE DIFFERENT

CATEGORY	WHAT IT MEANS	CAN BREAK THE APP?	CAN BE EXPLOITED?	NEEDS HUMAN REVIEW?	MAIN GOAL
BUGS	Defects that affect functionality	✗ YES	✗ NO	— SOMETIMES	Fix to improve reliability
VULNERABILITIES	Security issues that can be exploited	— SOMETIMES	✓ YES	— SOMETIMES	Fix to improve security
CODE SMELLS	Maintainability issues that increase tech debt	✓ NO	✓ NO	✓ NO	Refactor to improve maintainability
SECURITY HOTSPOTS	Potential security risks that need review	— SOMETIMES	— SOMETIMES	✓ YES	Review and decide (Fix or Accept)

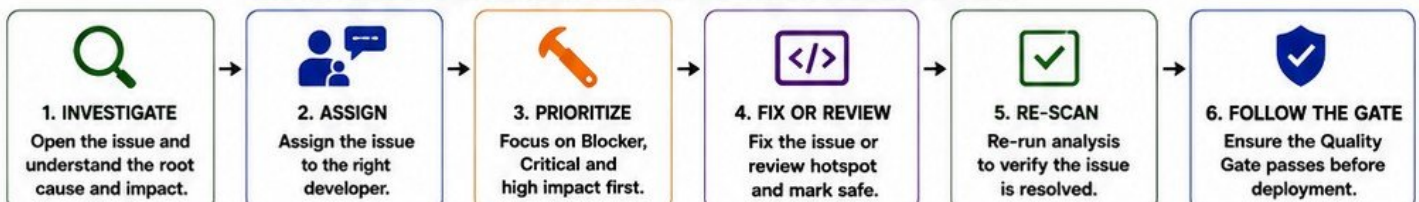
SEVERITY AND IMPACT

!	BLOCKER	Critical issue that must be fixed immediately. High risk and high impact.
!	CRITICAL	Very serious issue. Should be fixed as soon as possible.
!	MAJOR	Important issue that should be fixed to avoid future problems.
!	MINOR	Low impact issue. Fix when possible.
✓	INFO	Informational. No impact on code quality but helpful to know.

EXAMPLE FINDINGS

!	Bug: "Null pointer dereference" in UserService.java"
!	Vulnerability: "SQL Injection" in LoginController.java"
!	Code Smell: "This method has 52 lines of code (maximum is 30)"
!	Security Hotspot: "Use of hardcoded password detected"

WHAT A DEVOPS ENGINEER SHOULD DO WHEN ISSUES APPEAR



BEGINNER NOTE

SonarQube helps you build better and safer code by finding problems early. Combine it with your testing strategy. It is **not** a replacement for testing.

7. UNDERSTANDING RULES AND ISSUES



WHAT IS A SONARQUBE RULE?

A rule is a quality standard written by SonarSource or your team. It defines patterns to look for in code and what is considered a problem.



HOW RULES CREATE ISSUES

During analysis, SonarQube scans your code. When code violates a rule, SonarQube creates an issue and shows it in the dashboard for you to fix.

RULE SEVERITY

- BLOCKER** – Critical issue. Must be fixed.
- CRITICAL** – Very serious issue.
- MAJOR** – Important issue.
- MINOR** – Low impact. Should be fixed.
- INFO** – Informational. No impact.

ISSUE STATUS

- OPEN** – Issue is new and not fixed.
- CONFIRMED** – Valid issue. Needs to be fixed.
- RESOLVED** – Fixed in the code. Needs re-scan.
- CLOSED** – Issue is fixed and confirmed.
- WONTFIX** – Issue will not be fixed.
- FALSE POSITIVE** – Not a real issue.



FALSE POSITIVES

Sometimes a rule flags code that is actually correct and safe. Always verify before changing code or disabling rules.



RESOLVING AN ISSUE

Change the code to follow the rule. Re-run analysis. If the issue is gone, status becomes **RESOLVED** → **CLOSED**.



ACCEPTING AN ISSUE

If the issue is valid but you decide not to fix it (for a good reason), you can mark it as **WONTFIX** or **FALSE POSITIVE**.



WHY NOT DISABLE RULES?

Disabling rules hides problems. It increases technical debt, security risks and future maintenance cost. Fix the code. Do not hide the problem.

RULE → ANALYSIS → ISSUE → DEVELOPER ACTION FLOW



1. RULE

Rule defines what good code looks like.



2. ANALYSIS

SonarQube scans your code.



3. ISSUE

If rule is violated, an issue is created.



4. DEVELOPER ACTION

Developer fixes, accepts or marks as false positive.



5. RE-SCAN

Re-run analysis to verify resolution.



KEY TAKEAWAY

Rules help you keep code clean, secure and maintainable. Know the difference between fixing the code, accepting an issue and disabling a rule. Use rules to improve quality, not to bypass quality.



YouTube / veriqta



GitHub / veriqta



LinkedIn / veriqta



X / veriqta



Instagram / veriqta



Telegram / veriqta

8. QUALITY PROFILES

QUALITY PROFILES CONTROL THE RULES THAT SONARQUBE USES TO ANALYZE YOUR CODE.



WHAT A QUALITY PROFILE CONTROLS

- A Quality Profile is a collection of rules.
- It defines which rules are active, their severity and how code is analyzed.
- It does NOT change your code.



LANGUAGE-SPECIFIC PROFILES

- Each programming language has its own rules.
- Example: Java profile, Python profile, JavaScript profile, etc.



BUILT-IN PROFILES

- SonarQube provides built-in profiles for each language.
- These are a good starting point for most teams.



SONAR WAY

- The default built-in profile is called "Sonar way".
- It follows industry best practices for clean, secure and maintainable code.



ACTIVE RULES

- Rules can be Active or Inactive.
- Only Active rules are used during analysis.
- You can adjust the Severity of each rule (Blocker, Critical, Major, Minor, Info).
- Fewer active rules = less strict analysis.
- More active rules = stricter quality checks.

SEVERITY LEVELS

- ! **BLOCKER** – Must be fixed immediately
- ! **CRITICAL** – Very serious issue
- ! **MAJOR** – Important issue
- ! **MINOR** – Low impact issue
- ✓ **INFO** – Informational only



CREATING A CUSTOM PROFILE

1. Go to Quality Profiles.
2. Click "Create".
3. Choose a language.
4. Copy an existing profile or start from scratch.
5. Activate or deactivate rules.
6. Set rule severity.
7. Save your profile.



INHERITING PROFILES

- You can create a profile that inherits another profile.
- It keeps all parent rules and lets you adjust only what you need.
- This keeps your configuration consistent and easier to manage.



ASSIGNING PROFILES TO PROJECTS

1. Go to your project.
2. Click "Project Settings".
3. Select "Quality Profiles".
4. Choose the appropriate profile for the language.
5. Save changes.
6. Next analysis will use the new profile.



TIP

- Use built-in profiles in the beginning.
- Customize only when your team understands the impact.
- Review changes regularly.



COMMON MISTAKE

CHANGING ORGANIZATION-WIDE RULES WITHOUT UNDERSTANDING THEIR IMPACT

- Disabling important rules hides real problems.
- It increases technical debt.
- It creates security risks.
- It leads to poor quality software.
- ALWAYS understand a rule before you change it.
- Fix the code. Do not lower the standard.

HOW QUALITY PROFILES WORK IN SONARQUBE



1. QUALITY PROFILE
Defines the active rules and severity.



2. ANALYSIS
SonarQube analyzes your code using these rules.



3. ISSUES FOUND
Violations become issues in the project.



4. QUALITY GATE
Issues are evaluated against quality gate conditions.



5. ACTION
Fix issues, accept valid ones, and re-scan.



KEY TAKEAWAY

Quality Profiles let you control what "good code" means for your team. Use them wisely to keep your code clean, secure and maintainable.

9. QUALITY GATES

STOP BAD CODE BEFORE IT REACHES PRODUCTION



WHAT IS A QUALITY GATE?

A Quality Gate is a set of conditions that your code must meet before it can be accepted.

It evaluates analysis results and decides if the code is good enough to move forward.



WHY CI/CD PIPELINES NEED GATES

- Prevents bad code from going to production.
- Enforces quality standards automatically.
- Gives fast feedback to developers.
- Reduces technical debt and rework.
- Builds trust in your delivery pipeline.

PASSED vs FAILED GATES



PASSED

All conditions are met. The pipeline continues.



FAILED

One or more conditions are not met. The pipeline stops.



QUALITY GATE CONDITIONS

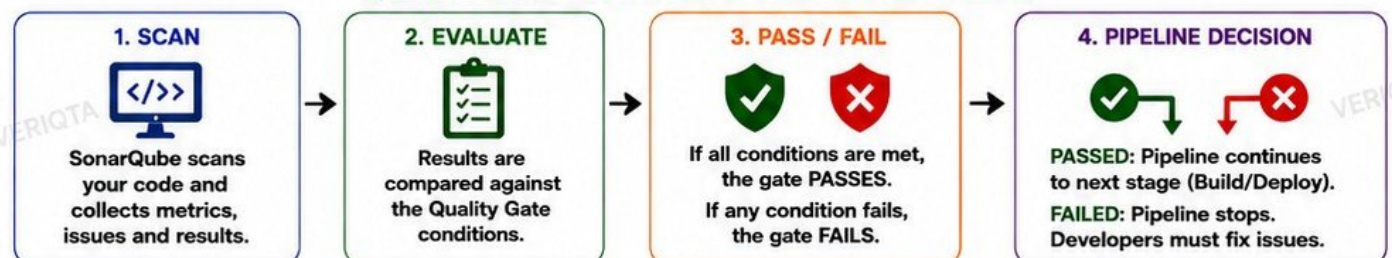
You define the rules that your code must satisfy.

Example conditions:
Coverage, Duplications, Reliability, Security, Maintainability, New Code conditions, and more.

COMMON QUALITY GATE CONDITIONS

CONDITION	WHAT IT MEASURES	EXAMPLE	WHY IT MATTERS
NEW CODE CONDITIONS	Applies rules only to the new or changed code.	• New Code bugs = 0 • New Code coverage $\geq 80\%$	Focuses on keeping new code clean without forcing legacy cleanup all at once.
COVERAGE	Measures how much of the code is covered by tests.	• Overall coverage $\geq 70\%$ • New Code coverage $\geq 80\%$	Higher coverage reduces risk of defects.
DUPLICATIONS	Measures duplicated or copied code.	• Duplication on New Code $< 3\%$ • Overall Duplication $< 10\%$	Less duplication = easier maintenance and fewer bugs.
RELIABILITY	Detects bugs that can cause crashes or errors.	• New Code bugs = 0 • Overall bugs = as low as possible	More reliable code means fewer production issues.
SECURITY	Finds vulnerabilities and security issues.	• New Code vulnerabilities = 0 • Security Rating = A	Protects applications and data from security risks.
MAINTAINABILITY	Checks code smells and maintainability issues.	• Maintainability Rating = A • New Code smells = 0	Cleaner code is easier to maintain and extend.

QUALITY GATE FLOW: HOW IT WORKS IN A PIPELINE



KEY TAKEAWAY

Quality Gates enforce your quality standards automatically. They protect your codebase, reduce risk and ensure only high-quality code moves forward. Define the right conditions and never lower the bar.

10. UNDERSTANDING CLEAN AS YOU CODE

FOCUS ON NEW CODE. IMPROVE LEGACY CODE GRADUALLY.



WHAT "NEW CODE" MEANS

- New code is the code added, modified or changed.
- SonarQube applies strict quality rules to this code.
- Legacy code still matters, but it is not the focus.



WHY FIXING ENTIRE LEGACY CODE IS DIFFICULT

- Huge volume of issues.
- High cost and time.
- High risk of breaking stable functionality.
- Slows down delivery.
- Teams lose motivation.



FOCUSING QUALITY CONTROLS ON CHANGED CODE

- Keeps the main branch clean.
- Prevents new issues from entering the codebase.
- Enables fast and safe delivery.
- Encourages developers to write better code.
- Technical debt stops increasing.



NEW CODE DEFINITIONS

- Previous Version Comparison (default).
- Number of Days.
- Reference Branch.
- Pull Request.
- Specific Analysis. Choose the option that matches your team's workflow.

QUALITY GATES ON NEW CODE

Quality Gates check only the new or changed code based on your definition. If the new code meets the conditions, the gate passes.

If not, the pipeline stops.



NEW CODE CONDITIONS



COVERAGE



DUPLICATIONS



RELIABILITY (BUGS)



SECURITY



MAINTAINABILITY (CODE SMELLS)

PREVENTING NEW TECHNICAL DEBT



Technical debt grows when new issues are introduced.

Clean as You Code prevents new debt while you pay down old debt over time.

WITHOUT CLEAN AS YOU CODE



TECHNICAL DEBT

New issues keep adding up.

WITH CLEAN AS YOU CODE



TECHNICAL DEBT

New issues are prevented. Debt reduces gradually.

VS

HOW IT WORKS IN A PIPELINE



1. CODE CHANGE

Developer writes and commits changes.



2. ANALYSIS

SonarQube scans only the new or changed code.



3. EVALUATE

Results are compared against the Quality Gate conditions.



4. PASS / FAIL

If new code meets all conditions, gate PASSES. If not, gate FAILS.



5. PIPELINE DECISION

Passed: Continue to next stage (Build/Deploy). Failed: Stop and fix issues.



BEGINNER NOTE

Protect new code first while improving legacy code gradually.

This keeps your codebase healthy, your pipeline fast, and your team productive.

11. CODE COVERAGE AND SONARQUBE

MEASURED. ANALYZED. ENFORCED.



WHAT CODE COVERAGE MEANS

- Code coverage measures how much of your code is executed by tests.
- It shows which parts of your code are tested.
- Higher coverage helps reduce the risk of hidden bugs.



SONARQUBE VERSUS TESTING TOOLS

- Testing tools run your tests and generate coverage data.
- SonarQube does not run tests.
- SonarQube imports the coverage report and uses it in analysis and Quality Gates.



HOW COVERAGE REPORTS REACH SONARQUBE

1. Testing tool runs your tests.
2. Coverage report is generated (e.g., XML format).
3. SonarScanner picks the report during analysis.
4. SonarQube imports and displays the coverage.



LINE COVERAGE

- Measures the percentage of code lines executed by tests.
- Example: 85% means 85% of code lines were run.
- Focuses on lines, not outcomes.



BRANCH COVERAGE

- Measures the percentage of branches/paths executed by tests.
- Ensures both true and false paths are tested.
- Stronger indicator of test quality than line coverage.



COVERAGE ON NEW CODE

- Quality Gates usually check coverage only on new or changed code.
- Prevents new code from being merged with low coverage.
- Legacy code can be improved gradually.



WHY 100% COVERAGE DOES NOT GUARANTEE GOOD SOFTWARE

- Tests may be weak or meaningless.
- Does not guarantee correct behavior.
- Does not check security, performance or design.
- Quality > Quantity.

COMMON MISSING-REPORT PROBLEMS



1. Report not generated – Tests did not produce a coverage file.
2. Wrong report path – SonarScanner cannot find the file.
3. Unsupported format – SonarQube supports specific formats (e.g., JaCoCo XML, LCOV).
4. Path mismatch – Report paths do not match project structure.
5. Empty or zero-byte file – Report is invalid.
6. Multiple reports conflict – Wrong file is picked up.
7. Not configured in scanner – Missing sonar.coverage.jacoco.xmlReportPaths (or equivalent).



COVERAGE FLOW: FROM TEST TO SONARQUBE

1. TEST



Run your unit/integration tests using a testing framework (JUnit, PyTest, NUnit, etc.).

2. COVERAGE REPORT



Testing tool generates a coverage report (e.g., JaCoCo XML).

3. SONARSCANNER



SonarScanner reads the coverage report and sends it to SonarQube during analysis.

4. SONARQUBE



SonarQube imports the coverage data and shows it in the dashboard and Quality Gates.



KEY TAKEAWAY

- Coverage helps you test more of your code, but good tests make good software.
- Always ensure coverage reports are generated and correctly imported.
- Use Quality Gates to enforce coverage on new code.



YouTube / veriQTA



GitHub / veriQTA



LinkedIn / veriQTA



X / veriQTA



Instagram / veriQTA



Telegram / veriQTA

12. SONARQUBE WITH GIT AND PULL REQUESTS



DEVELOPER WORKFLOW

Developers create changes in feature branches and open pull requests to propose those changes to the main branch.



FEATURE BRANCHES

Work happens in isolated feature branches. This keeps the main branch stable and production ready.



PULL REQUESTS

A pull request is opened to merge changes from the feature branch into the main branch. This triggers automated checks.



RUNNING ANALYSIS DURING CODE REVIEW

CI runs SonarQube analysis on the pull request. The analysis focuses on new or changed code.



DETECTING PROBLEMS BEFORE MERGE

Bugs, vulnerabilities, code smells and quality issues are found before the code is merged.



QUALITY GATE FEEDBACK

SonarQube evaluates the code against the Quality Gate. The result (Pass/Fail) is shown in the pull request.



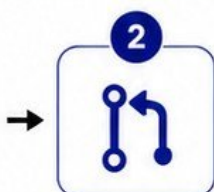
PREVENTING POOR-QUALITY CHANGES

If the Quality Gate fails, the pull request is blocked. This stops low-quality or risky code from reaching the main branch.

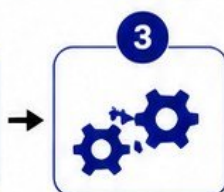
PRACTICAL FLOW



DEVELOPER
Writes code in feature branch



PULL REQUEST
Opens PR to merge changes



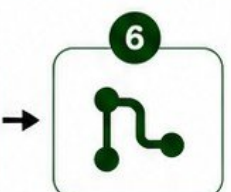
CI PIPELINE
Builds, runs tests and triggers scan



SONARQUBE
Analyzes code and checks quality gate



CODE REVIEW
Team reviews code and quality gate results



MERGE
Merge only if quality gate passes



WHEN QUALITY GATE PASSES

- Code meets quality standards
- Pull request can be merged
- Safe to reach the main branch
- Higher quality software



WHEN QUALITY GATE FAILS

- Issues must be fixed
- Pull request is blocked
- Prevents bad code from merging
- Keeps main branch clean and safe



BEGINNER NOTE

SonarQube in pull requests gives fast feedback to developers. Fix problems early, keep code clean, and deliver better software.

13. SONARQUBE WITH JENKINS



WHERE SONARQUBE FITS INTO JENKINS

SonarQube is part of your Jenkins pipeline. It analyzes code quality before deployment and ensures only clean, safe code moves forward.



JENKINS SONARQUBE INTEGRATION

- Install SonarQube Scanner plugin
- Add SonarQube server in Jenkins
- Create credentials (Token)
- Use the token in the pipeline securely



CREDENTIALS

Use a SonarQube user token with 'Execute Analysis' permission. Store it securely in Jenkins Credentials.



ANALYSIS STAGE

Jenkins runs the SonarQube Scanner to analyze the code. Results are sent to SonarQube Server.



QUALITY GATE STAGE

Jenkins checks the Quality Gate status (Pass/Fail) from SonarQube. This decides the next step in the pipeline.



WHEN QUALITY GATE FAILS

The pipeline is stopped. The build is marked as failed. Deployment is blocked. Developers must fix issues and push code again.

PIPELINE SEQUENCE



1 JENKINS

Pipeline starts



2 BUILD

Compile the code and prepare artifacts



3 TEST

Run unit & integration tests
Generate reports (e.g., coverage)



4 SONARQUBE

Analyze code quality, bugs, vulnerabilities, code smells



5 QUALITY GATE

Check Quality Gate status (Pass / Fail)



6 DEPLOY

Deploy to staging / production only if quality gate passes

WHAT HAPPENS WHEN QUALITY GATE FAILS



- Pipeline is marked as FAILED
- Deployment stage is skipped
- Developers get feedback in SonarQube
- Issues must be fixed in the code
- Code is pushed again and pipeline re-runs



QUALITY GATE FAILURE = PROTECTS YOUR CODEBASE FROM POOR QUALITY AND RISKS.

COMMON PIPELINE FAILURES

- ✗ Cannot find SonarQube server URL
- ✗ Invalid or expired authentication token
- ✗ SonarScanner not found in PATH
- ✗ Project key is missing or incorrect
- ✗ Network or firewall blocking Jenkins
- ✗ Analysis completed but Quality Gate not found
- ✗ Coverage report path is wrong or missing
- ✗ Insufficient permissions for the token

THE JENKINS + SONARQUBE PIPELINE IN ONE LINE

JENKINS (START)

BUILD

TEST

SONARQUBE (ANALYSIS)

QUALITY GATE (PASS / FAIL)

DEPLOY (IF PASS)



BEGINNER NOTE

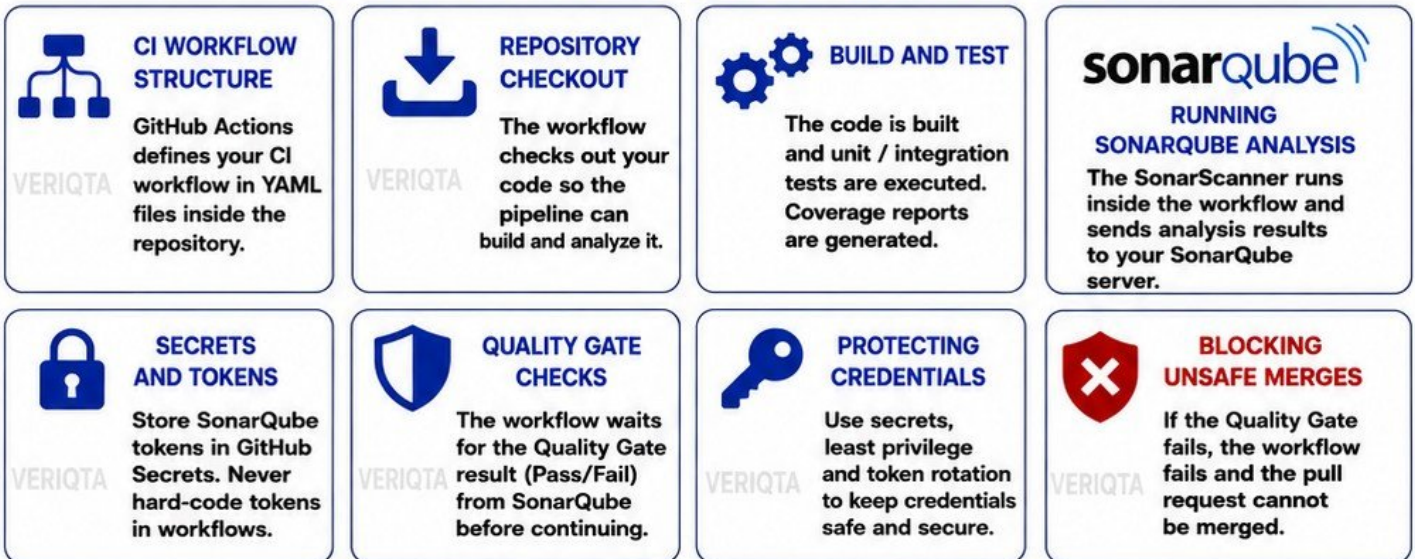
SonarQube in Jenkins helps you find problems early, keep code clean, and deliver better software. Make it a mandatory quality step in every pipeline.



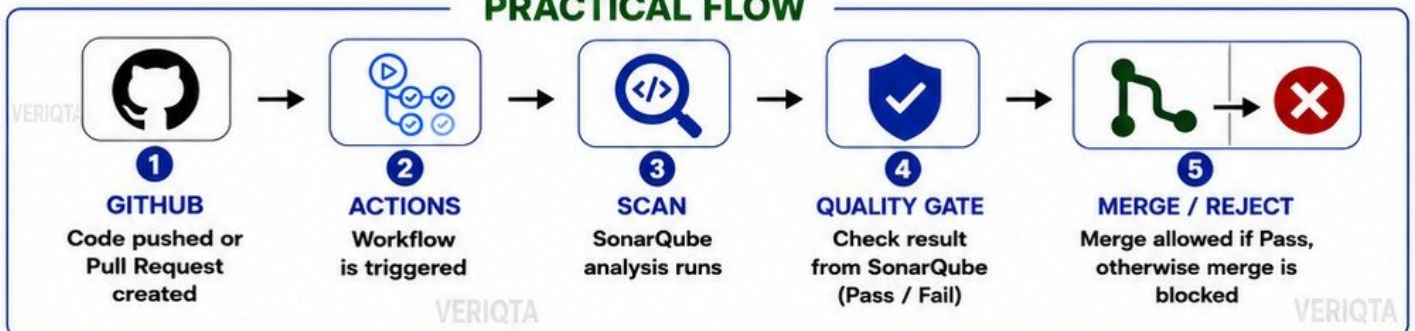
14. SONARQUBE WITH GITHUB ACTIONS

VERIQTA

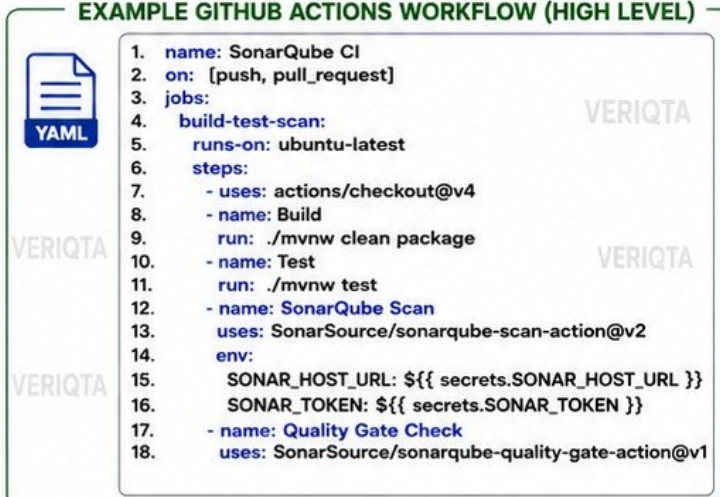
VERIQTA



PRACTICAL FLOW



EXAMPLE GITHUB ACTIONS WORKFLOW (HIGH LEVEL)



COMMON WORKFLOW FAILURES

- ! SonarQube server URL is incorrect or unreachable
- ! Invalid or expired SONAR_TOKEN
- ! Scanner cannot find the project key
- ! Coverage report path is wrong or missing
- ! Quality Gate check step times out
- ! Not enough permissions for the token
- ! Workflow runs but analysis is not published

WHY THIS MATTERS



Catching issues in Pull Requests prevents poor-quality code from reaching the main branch and keeps your codebase clean, safe and maintainable.



BEGINNER NOTE

SonarQube in GitHub Actions gives fast feedback to developers. Fix issues early, follow the Quality Gate and ship better software.

VERIQTA



15. AUTHENTICATION, TOKENS AND PERMISSIONS



USERS AND GROUPS

- SonarQube manages users and groups.
- Groups make it easy to manage permissions for teams.
- Assign users to groups and groups to projects.



PROJECT PERMISSIONS

- Control what users and groups can do in a project.
- Examples:
 - Administer
 - Execute Analysis
 - Browse
 - Configure
- Assign only what is needed.



ANALYSIS PERMISSIONS

- Required permission: "Execute Analysis"
- Allows a user or token to run analysis.
- Without it, analysis will fail with a permission error.



TOKENS

- Tokens are secure credentials used by CI/CD tools to authenticate.
- Each token has a name, an expiration date and specific permissions.
- Tokens act as the user who generated them.



WHY CI/CD SHOULD USE TOKENS INSTEAD OF PASSWORDS

- More secure: tokens can be scoped and limited.
- Easier to rotate and revoke.
- No need to store real user passwords.
- Prevents account lockouts and unauthorized access.



SECRET STORAGE

- Store tokens in your CI/CD secret manager.
- Examples: Jenkins Credentials, GitHub Secrets, GitLab CI/CD Variables, Vault, AWS Secrets Manager.
- Never commit secrets to source code.



TOKEN ROTATION

- Rotate tokens regularly.
- Use short expiration periods.
- Revoke unused or old tokens.
- Create new tokens and update CI/CD secrets.



LEAST PRIVILEGE

- Give only the minimum permissions required.
- Example: CI/CD usually needs only "Execute Analysis".
- Avoid using Admin tokens in automation.



BEST PRACTICES

- Use dedicated tokens for CI/CD.
- One token per purpose.
- Monitor token usage.
- Disable tokens that are no longer in use.



SECURITY REMINDER

- ✗ **NEVER HARD-CODE SONARQUBE TOKENS IN REPOSITORIES OR PIPELINE FILES.**
 - Hard-coded secrets can be exposed to anyone.
 - Exposed tokens can be misused to run analysis or access data.
 - Always use environment variables or secret storage provided by your CI/CD platform.

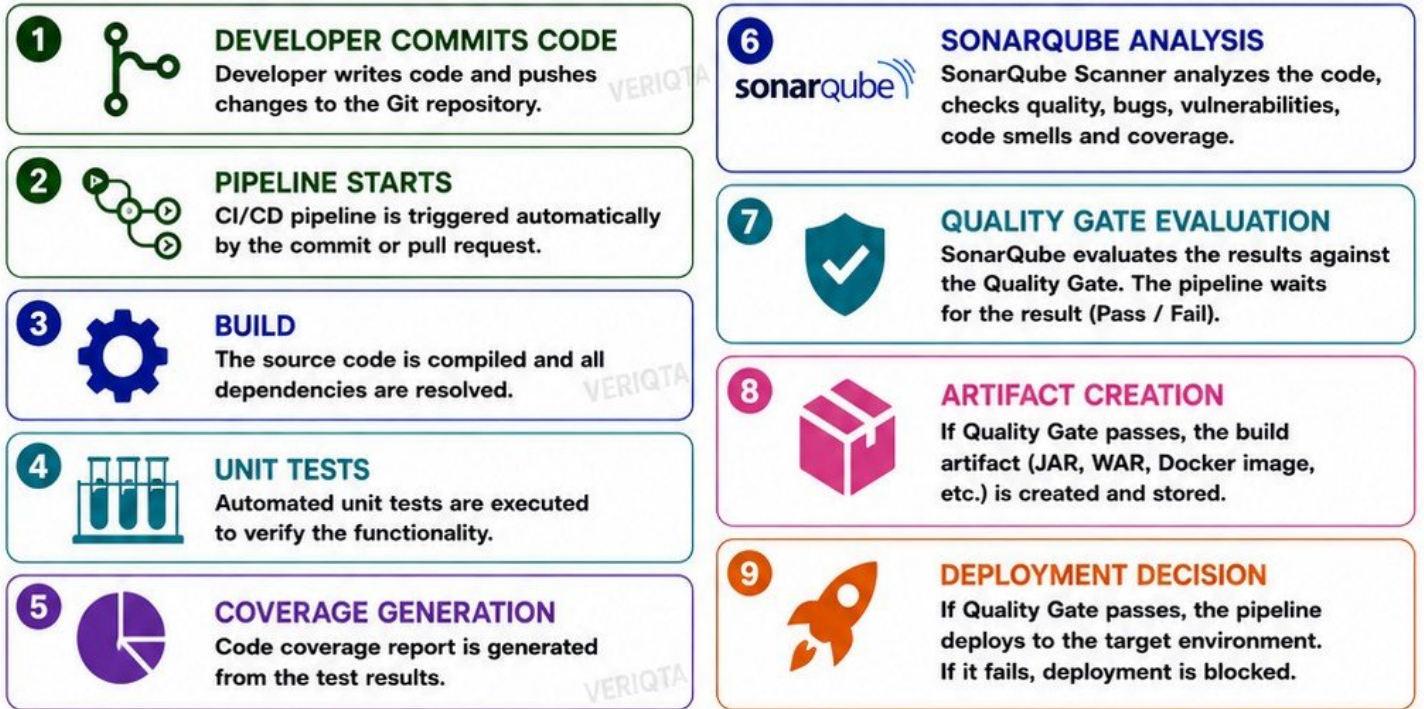
AUTHENTICATION FLOW IN CI/CD



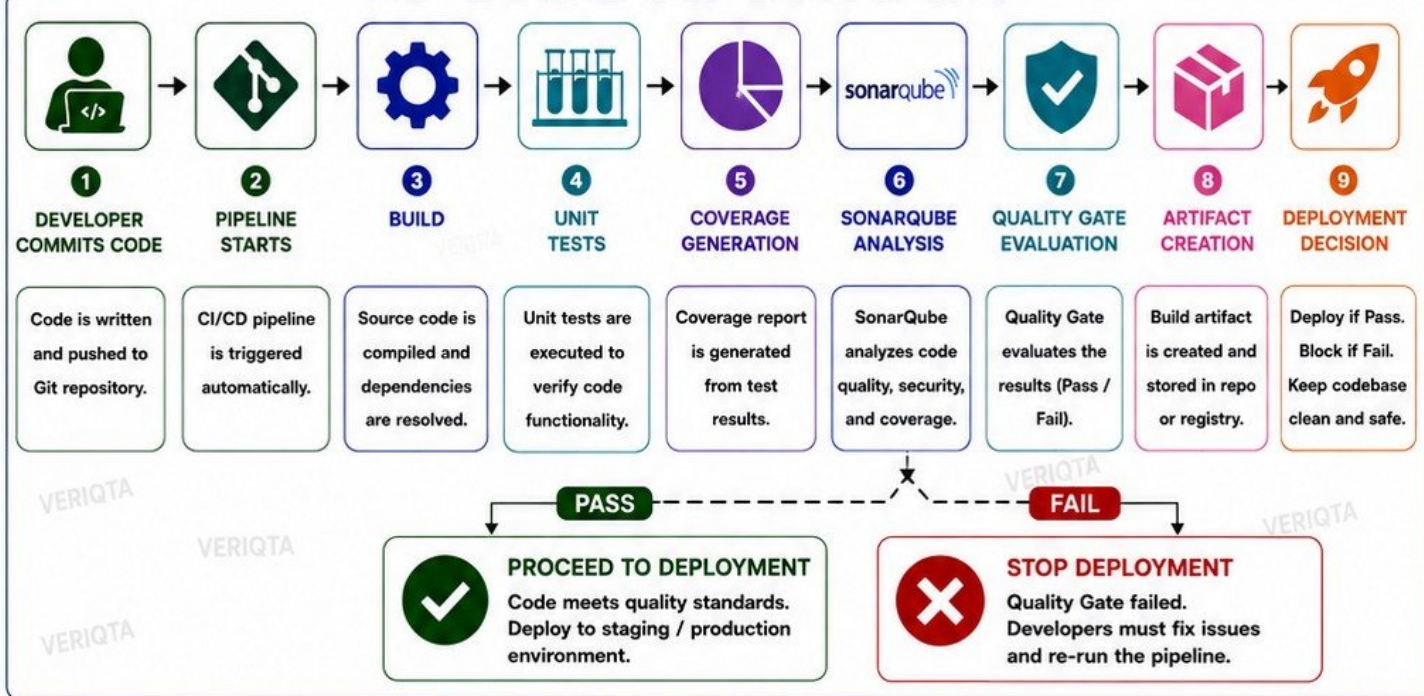
BEGINNER NOTE

Authentication, tokens and permissions protect your projects and data. Use tokens, limit access and keep your pipeline secure.

16. SONARQUBE IN A REAL CI/CD PIPELINE



COMPLETE CI/CD ARCHITECTURE DIAGRAM



PRODUCTION NOTE

Run quality checks **BEFORE** expensive deployment stages. Fix issues early, save time, reduce risk and deliver better software.

17. TROUBLESHOOTING FAILED SONARQUBE ANALYSIS



SONARQUBE ANALYSIS FAILS WHEN THE SCANNER CANNOT CONNECT, IF CONFIGURATION IS WRONG, OR WHEN REQUIRED FILES ARE MISSING.

COMMON REASONS FOR FAILED ANALYSIS

1



AUTHENTICATION FAILURE

Invalid or expired token, wrong credentials, or insufficient permissions.

2



INVALID PROJECT KEY

Project key does not exist or is misspelled in the configuration.

3



SONARQUBE SERVER UNREACHABLE

Scanner cannot connect to the SonarQube server (server is down or URL is incorrect).

4



SCANNER NOT FOUND

SonarScanner is not installed or not available in the system PATH.

5



INCORRECT SOURCE PATH

Sources path is wrong or files are not where the scanner expects.

6



UNSUPPORTED RUNTIME

Unsupported Java version or runtime environment for SonarScanner.

7



MISSING COVERAGE REPORTS

Coverage report path is missing, incorrect, or the file was not generated.

8



NETWORK AND DNS PROBLEMS

Firewall, proxy, DNS issues or blocked network connections.

READING SCANNER LOGS



- Scanner logs show exactly where and why the failure happened.
- Look for **ERROR** lines (in red) to identify the main problem.
- Logs include details about connection, authentication, files, and analysis.
- Always check the full log, not just the last few lines.

DEBUGGING SEQUENCE: FOLLOW THIS ORDER EVERY TIME

1



ERROR

Identify the error message from the pipeline or scanner.

2



LOGS

Read the full scanner logs. Look for the exact failure reason.

3



CONNECTIVITY

Check network connection to SonarQube server.

4



CREDENTIALS

Verify token, permissions, and authentication settings.

5



CONFIGURATION

Check project key, source path, URL, Java version, and other settings.

6



RETRY

Fix the issue and re-run the analysis. Validate success.

QUICK CHECKLIST



- ✓ Correct SonarQube server URL and port
- ✓ Valid project key
- ✓ Valid authentication token
- ✓ Token has 'Execute Analysis' permission
- ✓ SonarScanner is installed and in PATH
- ✓ Java version is supported
- ✓ Source path points to the correct code location
- ✓ Coverage report path is correct (if used)
- ✓ Network connectivity to server is working
- ✓ No firewall or proxy blocking the connection



IMPORTANT REMINDER

Do not ignore or bypass SonarQube analysis failures. Fix the root cause to keep your code quality, security, and deployments safe.

18. TROUBLESHOOTING QUALITY GATE FAILURES



ANALYSIS SUCCEEDS BUT PIPELINE FAILS

This means the Quality Gate did not pass. You must find the reason and fix the issue.



FINDING THE FAILED CONDITION

Go to the project dashboard → Quality Gate. Click "Conditions" to see which rule failed and by how much.

COMMON REASONS QUALITY GATES FAIL



NEW BUGS

Bugs were introduced in new or changed code.



COVERAGE BELOW THRESHOLD

Code coverage is lower than the required limit.



DUPLICATED CODE

Too much code is duplicated across the codebase.



SECURITY FINDINGS

New vulnerabilities or security hotspots were detected.



OTHER CONDITIONS

Maintainability, reliability, complexity or issues count exceeded limits.



FIX VERSUS BYPASS

FIX (RECOMMENDED)

Address the real issue in the code. Commit the changes and re-run the pipeline.

BYPASS (NOT RECOMMENDED)

For emergency only and with approval. Extends technical debt and risk.



WHY BLINDLY OVERRIDING QUALITY GATES IS DANGEROUS

- Poor code reaches production
- More bugs and incidents
- Security risks increase
- Higher maintenance cost
- Loss of trust in the pipeline
- Technical debt grows faster

TROUBLESHOOTING CHECKLIST

- ✓ Open project dashboard
- ✓ Check Quality Gate status
- ✓ Review failed condition
- ✓ Read the issue details
- ✓ Identify the root cause
- ✓ Fix the code
- ✓ Commit and push changes
- ✓ Re-run the analysis
- ✓ Confirm Quality Gate passes

CORRECT WORKFLOW



DEVELOPER



FIX

Fix the issues



COMMIT

Commit changes



RESCAN

Run SonarQube analysis again



GATE PASS

Quality Gate passes



PIPELINE CONTINUES

Deploy safely



YouTube



GitHub



LinkedIn



x



Instagram



Telegram



VERIQTA

19. BEGINNER PRODUCTION PROJECT

Build a complete DevOps project with SonarQube integrated into the pipeline.



PROJECT OBJECTIVE

Create a full CI/CD workflow that enforces code quality using SonarQube Quality Gates before deployment.

WHAT YOU WILL BUILD

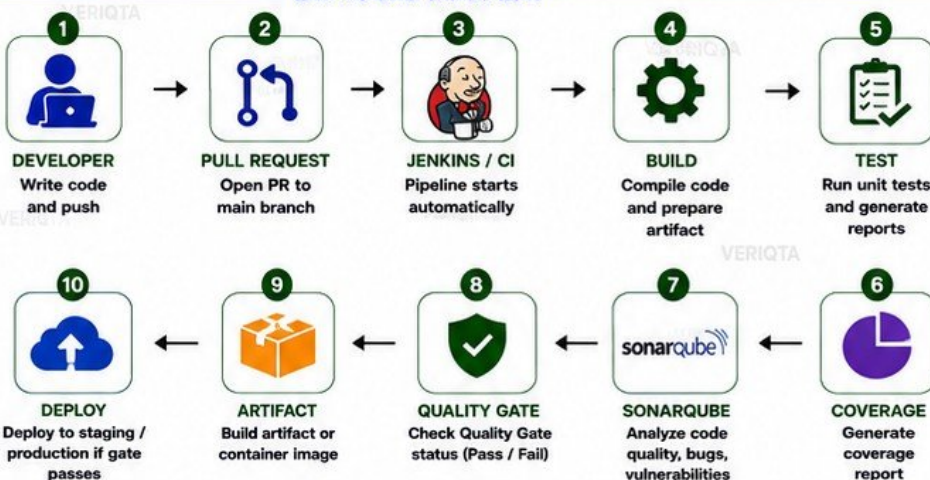
- ✓ Application source code
- ✓ Git repository
- ✓ Pull request workflow
- ✓ Jenkins or GitHub Actions
- ✓ Build and unit tests
- ✓ Coverage report
- ✓ SonarQube analysis
- ✓ Quality Gate validation
- ✓ Artifact / Container image
- ✓ Deployment to staging
- ✓ Pipeline fails if Quality Gate fails



SUCCESS CRITERIA

Code is built, tested, scanned and deployed only when the Quality Gate passes.

END-TO-END WORKFLOW



IF QUALITY GATE FAILS: » Pipeline is stopped » Deployment is blocked » Developers fix issues and push again



YouTube



GitHub



LinkedIn



x



Instagram



Telegram

VERIQTA

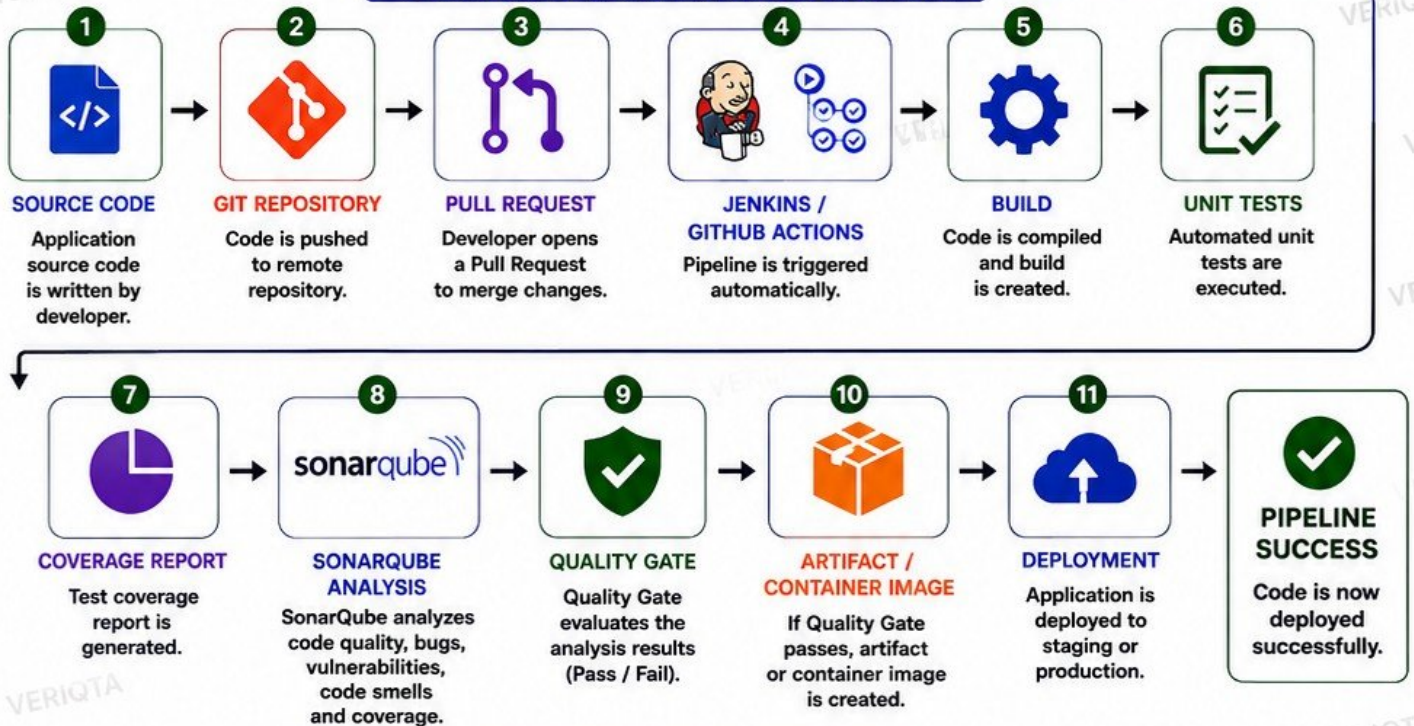


VERIQTA

19. BEGINNER PRODUCTION PROJECT

BUILD A COMPLETE SONARQUBE-ENABLED CI/CD WORKFLOW

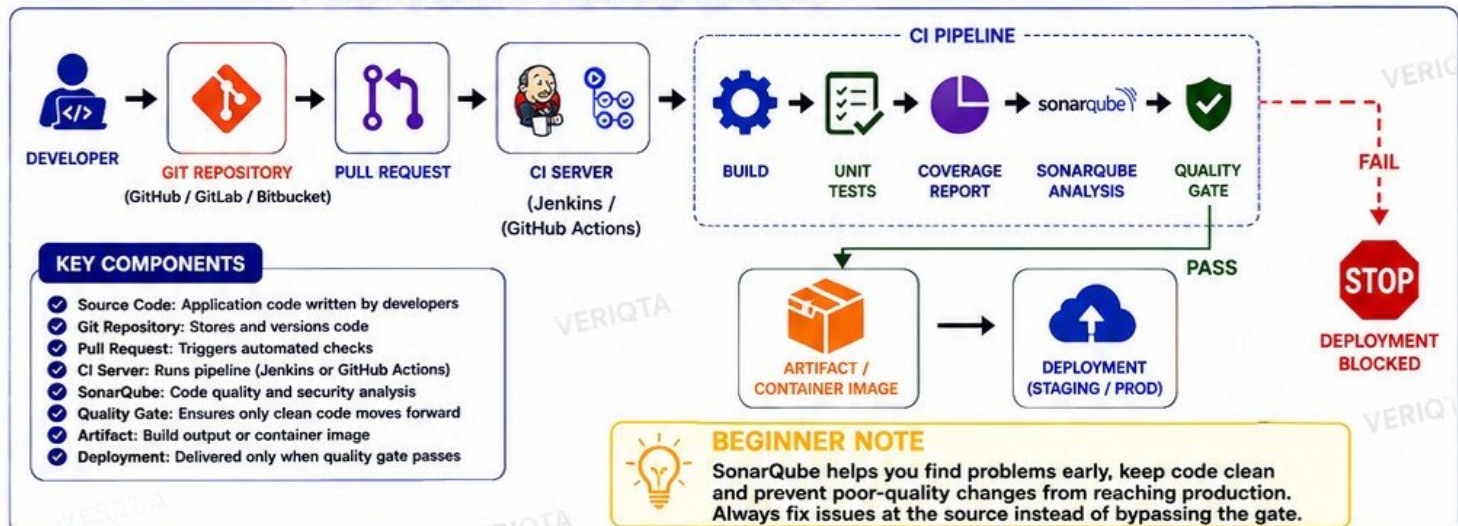
COMPLETE CI/CD PIPELINE WORKFLOW



WHAT HAPPENS WHEN QUALITY GATE FAILS



FINAL CI/CD ARCHITECTURE DIAGRAM



20. SONARQUBE DEVOPS TROUBLESHOOTING LAB



FIVE PRACTICAL BEGINNER SCENARIOS

Use this lab to diagnose and fix the most common SonarQube issues in CI/CD pipelines.

1



SonarScanner cannot connect to the SonarQube server.

2



The pipeline returns "Not authorized. Please check the user token."

3



Analysis succeeds, but code coverage shows 0%.

4



SonarQube analysis completes, but the Quality Gate fails.

5



A developer wants to bypass the Quality Gate to deploy urgently.

TROUBLESHOOTING GUIDE FOR EACH SCENARIO

STEP	WHAT HAPPENED?	WHAT SHOULD YOU CHECK FIRST?	LIKELY ROOT CAUSE?	HOW WOULD YOU TROUBLESHOOT IT?	CORRECT FIX	WHAT TO AVOID DOING
WHAT HAPPENED?	Scanner fails to reach SonarQube server.	Pipeline returns "Not authorized. Please check the user token."	Analysis works, but coverage in SonarQube is 0%.	SonarQube analysis completes, but Quality Gate fails.	Developer wants to bypass the Quality Gate to deploy urgently.	
CHECK FIRST	Network connectivity and server URL.	Token value and permissions.	Coverage report path and format.	Quality Gate conditions and failed metrics.	The real reason behind urgency and the risk.	
ROOT CAUSE	Wrong URL, server down, firewall, DNS issue, or port blocked.	Invalid, expired, or missing token. Insufficient permissions.	Coverage report not generated, wrong path, or unsupported format.	New issues, low coverage, vulnerabilities, duplicated code, or code smells.	Pressure to deploy without fixing quality issues.	
TROUBLESHOOT STEP-BY-STEP	1. Ping the server 2. Check URL & port 3. Check firewall 4. Check server logs 5. Retry connection	1. Verify token 2. Check permissions 3. Ensure token is in Jenkins/GitHub Secrets 4. Retry scan	1. Verify test execution 2. Check report path 3. Confirm report format (XML) 4. Validate paths in scanner config	1. Open project dashboard 2. Check failed conditions 3. Review issues 4. Check new code metrics	1. Discuss the real risk with the team 2. Review failures 3. Propose quick fixes if possible	
CORRECT FIX	Fix URL or network issue. Ensure SonarQube server is reachable.	Use a valid token with proper permissions. Store it securely.	Generate coverage report correctly and provide the correct path to SonarQube.	Fix the underlying issues in the code. Improve coverage or quality metrics.	Fix the issue or get proper approval with documented risk.	
WHAT TO AVOID	Do not ignore connection errors or assume the server is down.	Do not hard-code tokens or share them publicly.	Do not bypass tests or fake coverage reports.	Do not lower Quality Gate standards without team agreement.	Do not bypass the Quality Gate just to deploy faster.	

FINAL ARCHITECTURE EXERCISE: COMPLETE DEVOPS WORKFLOW



IF QUALITY GATE PASSES

Pipeline continues → Artifact created → Application deployed successfully.



IF QUALITY GATE FAILS

Pipeline stops → Deployment blocked → Fix issues → Re-run pipeline.



KEY TAKEAWAY

SonarQube helps you find problems early, keep code clean, and prevent bad code from reaching production. Always fix the issue, don't skip the gate.

